

NEURAL NETWORKS AND DEEP LEARNING (PE-5 18CSE23)

UNIT - V

Recurrent Neural Networks : Backpropagation through time (BPTT), Vanishing and Exploding Gradients, Truncated BPTT, GRU, LSTMs, Encoder-Decoder Models, Attention Mechanism, Attention over images

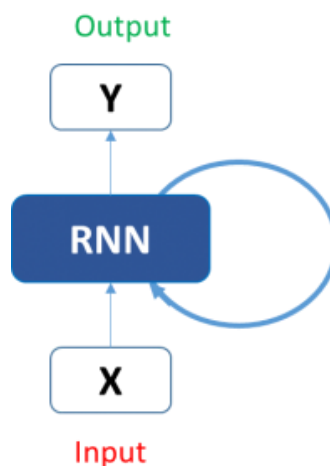
Recurrent Neural Networks:

- Recurrent Neural Network (RNN) is a neural network model proposed in the 80's for modelling time series.
- The structure of the network is similar to feedforward neural network, with the distinction that it allows a recurrent hidden state whose activation at each time is dependent on that of the previous time(cycle).

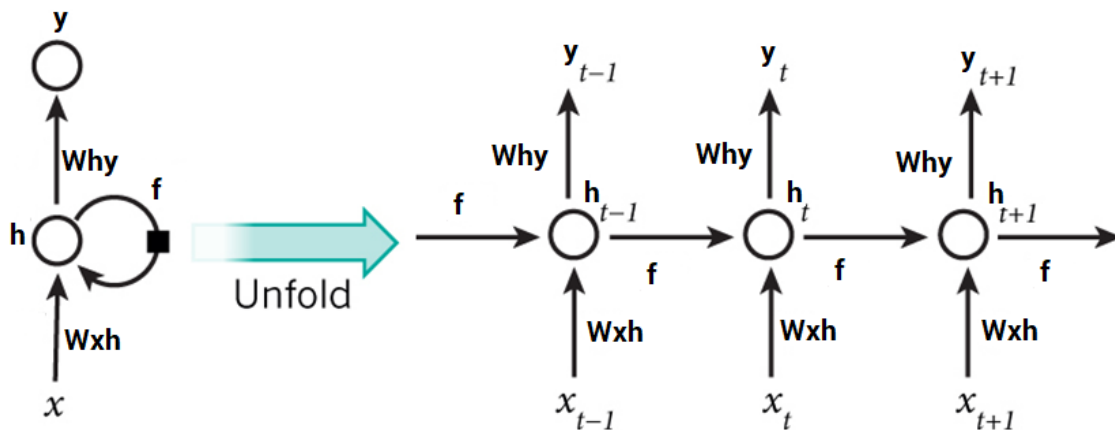
Take an example of sequential data, which can be the stock market's data for a particular stock. A simple machine learning model or an Artificial Neural Network may learn to predict the stock prices based on a number of features: the volume of the stock, the opening value etc. While the price of the stock depends on these features, it is also largely dependent on the stock values in the previous days. In fact for a trader, these values in the previous days (or the trend) is one major deciding factor for predictions.

In the conventional feed-forward neural networks, all test cases are considered to be independent. That is when fitting the model for a particular day, there is no consideration for the stock prices on the previous days.

This dependency on time is achieved via Recurrent Neural Networks. A typical RNN looks like:



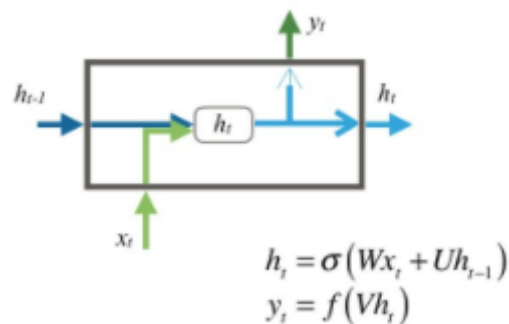
This may be intimidating at first sight, but once unfolded, it looks a lot simpler:



Now it is easier for us to visualize how these networks are considering the trend of stock prices, before predicting the stock prices for today. Here every prediction at time t (h_t) is dependent on all previous predictions and the information learned from them.

RNNs can solve our purpose of sequence handling to a great extent but not entirely. We want our computers to be good enough to write Shakespearean sonnets. Now RNNs are great when it comes to short contexts, but in order to be able to build a story and remember it, we need our models to be able to understand and remember the context behind the sequences, just like a human brain. This is not possible with a simple RNN.

Simple RNN



- The time recurrence is introduced by relation for hidden layer activity h_t with its past hidden layer activity h_{t-1} .
- This dependence is nonlinear because of using a logistic function.

Limitations of RNNs

Recurrent Neural Networks work just fine when we are dealing with short-term dependencies. That is when applied to problems like:

The colour of the sky is _____.

RNNs turn out to be quite effective. This is because this problem has nothing to do with the context of the statement. The RNN need not remember what was said before this, or what was its meaning, all they need to know is that in most cases the sky is blue. Thus the prediction would be:

The colour of the sky is blue.

However, vanilla RNNs fail to understand the context behind an input. Something that was said long before, cannot be recalled when making predictions in the present. Let's understand this as an example:

I spent 20 long years working for the under-privileged kids in Spain. I then moved to Africa.

.....

I can speak fluent _____.

Here, we can understand that since the author has worked in Spain for 20 years, it is very likely that he may possess a good command of Spanish. But, to make a proper prediction, the RNN needs to remember this context. The relevant information may be separated from the point where it is needed, by a huge load of irrelevant data. This is where a Recurrent Neural Network fails!

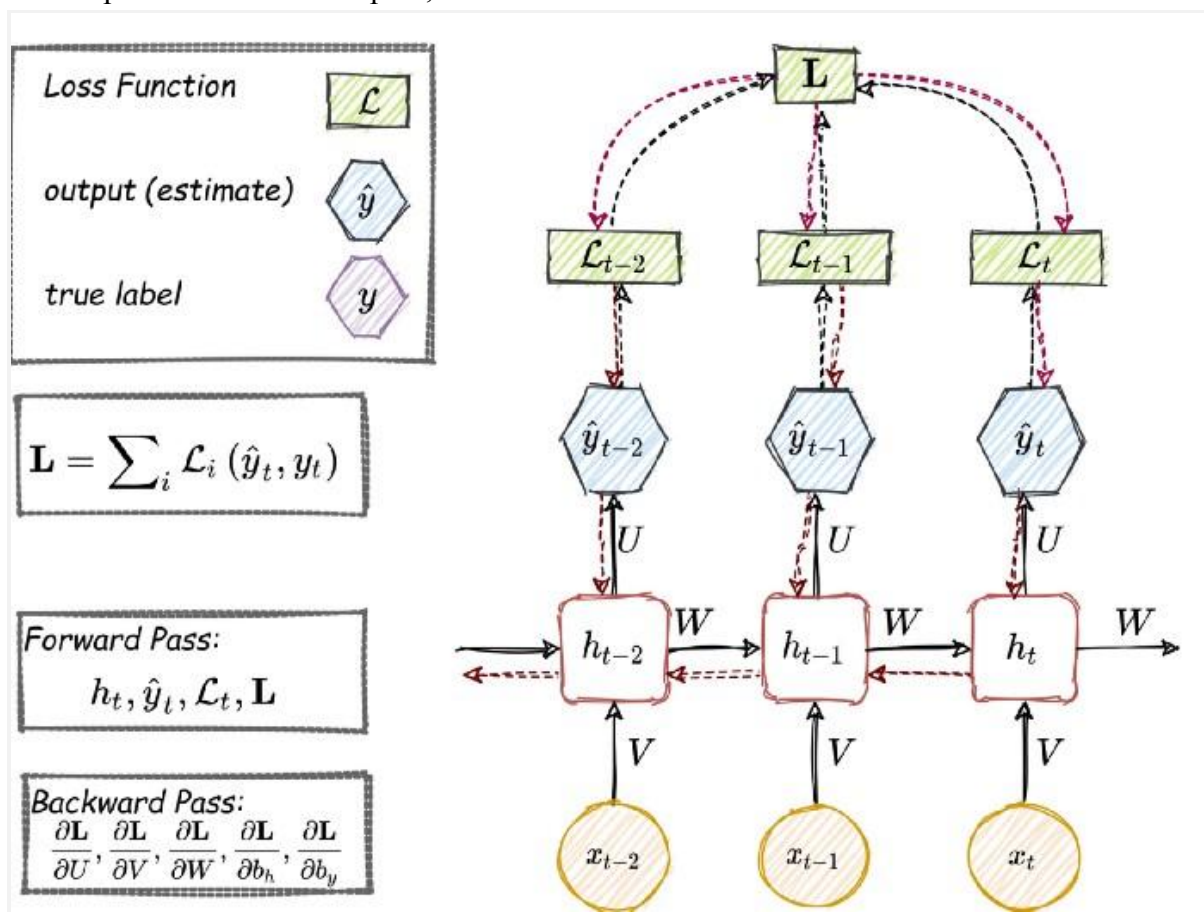
The reason behind this is the problem of Vanishing Gradient. In order to understand this, you'll need to have some knowledge about how a feed-forward neural network learns. We know that for a conventional feed-forward neural network, the weight updating that is applied on a particular layer is a multiple of the learning rate, the error term from the previous layer and the input to that layer. Thus, the error term for a particular layer is somewhere a product of all previous layers' errors. When dealing with activation functions like the sigmoid function, the small values of its derivatives (occurring in the error function) gets multiplied multiple times as we move towards the starting layers. As a result of this, the gradient almost vanishes as we move towards the starting layers, and it becomes difficult to train these layers.

A similar case is observed in Recurrent Neural Networks. RNN remembers things for just small durations of time, i.e. if we need the information after a small time it may be reproducible, but once a lot of words are fed in, this information gets lost somewhere. This issue can be resolved by applying a slightly tweaked version of RNNs – the Long Short-Term Memory Networks.

Backpropagation through time (BPTT)

Training an RNN is done by defining a loss function (L) that measures the error between the true label and the output, and minimizes it by using forward pass and backward pass. The following simple RNN architecture summarizes the entire backpropagation through time idea.

For a single time step, the following procedure is done: first, the input arrives, then it processes through a hidden layer/state, and the estimated label is calculated. In this phase, the loss function is computed to evaluate the difference between the true label and the estimated label. The total loss function, L , is computed, and by that, the forward pass is finished. The second part is the backward pass, where the various derivatives are calculated.



The training of RNN is not trivial, as we backpropagate gradients **through layers** and also **through time**. Hence, in each time step we have to sum up all the previous contributions until the current one, as given in the equation:

$$\frac{\partial \mathbf{L}}{\partial \mathbf{W}} = \sum_{i=0}^T \frac{\partial \mathcal{L}_i}{\partial \mathbf{W}} \propto \sum_{i=0}^T \left(\prod_{i=k+1}^y \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial \mathbf{W}}$$

In this equation, the contribution of a state at time step **k** to the gradient of the entire loss function **L**, at time step **t=T** is calculated. The challenge during the training is in the ratio of the hidden state:

$$\frac{\partial \mathbf{L}}{\partial \mathbf{W}} \propto \sum_{i=0}^T \left(\prod_{i=k+1}^y \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial \mathbf{W}}$$

- The unfolded recurrent neural network can be seen as a deep neural network, except that the recurrent weights are tied. To train it we can use a modification of the BP algorithm that works on sequences in time - backpropagation through time (BPTT).
- For each training epoch : start by training on shorter sequences, and then train on progressively longer sequences until the length of max sequence (1,2,.....N-1,N).
- For each length of sequence k : unfold the network into a normal feedforward network that has k hidden layers.
- Proceed with a standard BP algorithm.

The Vanishing and Exploding Gradients Problem

Two common problems that occur during the backpropagation of time-series data are the vanishing and exploding gradients. The equation above has two problematic cases:

- 
1. **Vanishing gradient** $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 < 1$
 2. **Exploding gradient** $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 > 1$

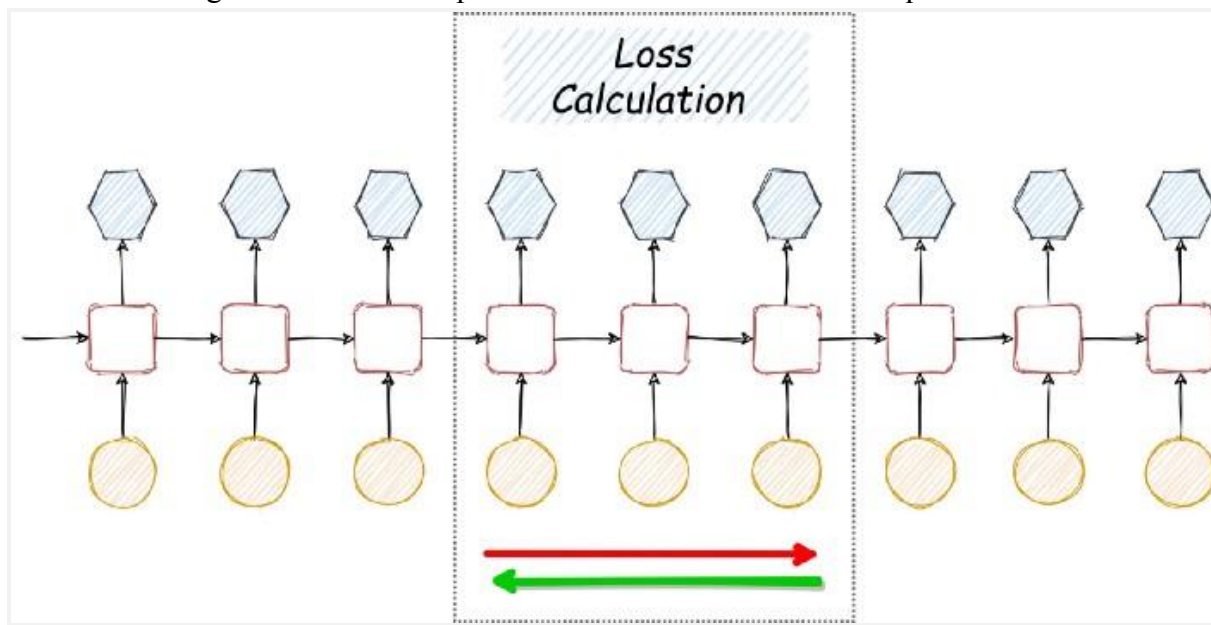
In the first case, the term goes to zero exponentially fast, which makes it difficult to learn some long period dependencies. This problem is called the **vanishing gradient**. In the second

case, the term goes to infinity exponentially fast, and their value becomes a NaN due to the unstable process. This problem is called the *exploding gradient*.

Truncated Backpropagation Through Time (Truncated BPTT).

The following “trick” tries to overcome the vanishing gradient problem by considering a moving window through the training process. It is known that in the backpropagation training scheme, there are a forward pass and a backward pass through the **entire sequence** to compute the loss and the gradient. By taking a window, we also improve the training performance from the training duration aspect- where we shortcut it.

This window is called a “*chunk*”. During the backpropagation process, we run forward and backward through this chunk of a specific size instead of the entire sequence.



The Truncated BPTT is much faster than the simple BPTT, and also less complex because we don't make the contribution of the gradients from faraway steps. The minus of this approach is that dependencies of longer than the chunk length, are not taught during the training process. Another disadvantage is the detection of the vanishing gradients. From looking at the learning curve one can assume that the gradient vanishes, but, maybe the task itself is difficult.

For the vanishing gradient problem, many other approaches have been suggested, to mention a few of them:

- Using ReLU activation function.
- Long-Short Term Memory (LSTM) architecture, where the forget gate might help.
- Initialize the weight matrix, $\mathbf{W}$, with an orthogonal matrix, and use this through the entire training (multiplications of orthogonal matrices doesn't explode or vanish).

One of the main problems of BPTT is the high cost of a single parameter update, which makes it impossible to use a large number of iterations.

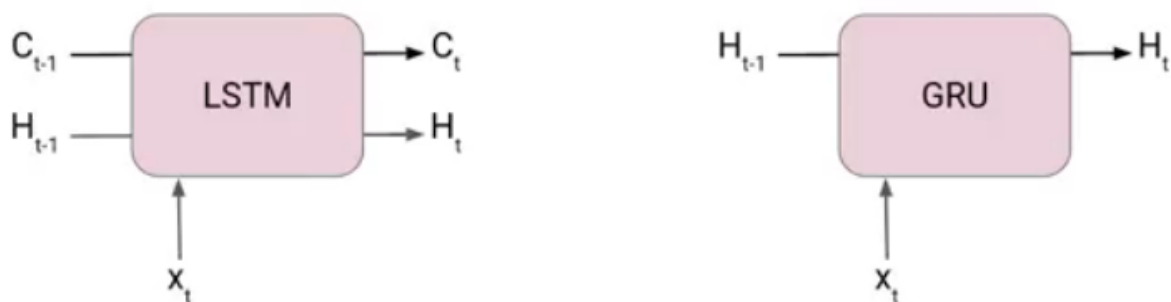
For instance, the gradient of an RNN on sequences of length 1000 costs the equivalent of a forward and a backward pass in a neural network that has 1000 layers.

Truncated BPTT processes the sequence on timestep at a time, and every T_1 timesteps it runs BPTT for T_2 timesteps, so a parameter update can be cheap if T_2 is small.

Truncated backpropagation is arguably the most practical method for training RNNs.

GRU (Gated Recurrent Unit)

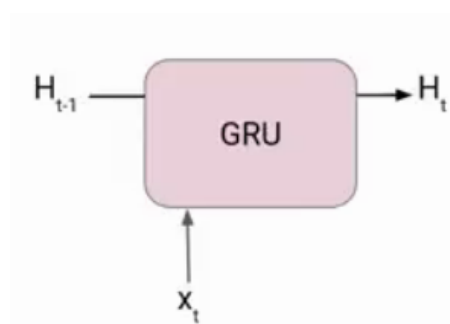
GRU or Gated recurrent unit is an advancement of the standard RNN i.e recurrent neural network. It was introduced by Kyunghyun Cho et al in the year 2014. GRUs are very similar to Long Short Term Memory (LSTM). Just like LSTM, GRU uses gates to control the flow of information. They are relatively new as compared to LSTM. This is the reason they offer some improvement over LSTM and have simpler architecture.



Another Interesting thing about GRU is that, unlike LSTM, it does not have a separate cell state (C_t). It only has a hidden state (H_t). Due to the simpler architecture, GRUs are faster to train.

The architecture of Gated Recurrent Unit

Now let's understand how GRU works. Here we have a GRU cell which is more or less similar to an LSTM cell or RNN cell.



At each timestamp t , it takes an input X_t and the hidden state H_{t-1} from the previous timestamp $t-1$. Later it outputs a new hidden state H_t which is again passed to the next timestamp.

Now there are primarily two gates in a GRU as opposed to three gates in an LSTM cell. The first gate is the Reset gate and the other one is the update gate.

Reset Gate (Short term memory)

The Reset Gate is responsible for the short-term memory of the network i.e the hidden state (H_t). Here is the equation of the Reset gate.

$$r_t = \sigma(x_t * U_r + H_{t-1} * W_r)$$

If you remember from the LSTM gate equation it is very similar to that. The value of r_t will range from 0 to 1 because of the sigmoid function. Here U_r and W_r are weight matrices for the reset gate.

Update Gate (Long Term memory)

Similarly, we have an Update gate for long-term memory and the equation of the gate is shown below.

$$u_t = \sigma(x_t * U_u + H_{t-1} * W_u)$$

The only difference is of weight metrics i.e U_u and W_u .

- How GRU Works

Now let's see the functioning of these gates. To find the Hidden state H_t in GRU, it follows a two-step process. The first step is to generate what is known as the candidate hidden state. As shown below

Candidate Hidden State

$$\hat{H}_t = \tanh(x_t * U_g + (r_t \circ H_{t-1}) * W_g)$$

It takes in the input and the hidden state from the previous timestamp $t-1$ which is multiplied by the reset gate output r_t . Later passed this entire information to the tanh function, the resultant value is the candidate's hidden state.

$$\hat{H}_t = \tanh(x_t * U_g + (r_t \circ H_{t-1}) * W_g)$$

The most important part of this equation is how we are using the value of the reset gate to control how much influence the previous hidden state can have on the candidate state.

If the value of r_t is equal to 1 then it means the entire information from the previous hidden state H_{t-1} is being considered. Likewise, if the value of r_t is 0 then that means the information from the previous hidden state is completely ignored.

Hidden state

Once we have the candidate state, it is used to generate the current hidden state H_t . It is where the Update gate comes into the picture. Now, this is a very interesting equation, instead of using a separate gate like in LSTM in GRU we use a single update gate to control both the historical information which is H_{t-1} as well as the new information which comes from the candidate state.

$$H_t = u_t \circ H_{t-1} + (1 - u_t) \circ \hat{H}_t$$

Now assume the value of u_t is around 0 then the first term in the equation will vanish which means the new hidden state will not have much information from the previous hidden state. On the other hand, the second part becomes almost one that essentially means the hidden state at the current timestamp will consist of the information from the candidate state only.

$$H_t = u_t \circ H_{t-1} + (1 - u_t) \circ \hat{H}_t$$

Similarly, if the value of u_t is on the second term will become entirely 0 and the current hidden state will entirely depend on the first term i.e the information from the hidden state at the previous timestamp $t-1$.

$$H_t = u_t \circ H_{t-1} + (1 - u_t) \circ \hat{H}_t$$

Hence we can conclude that the value of u_t is very critical in this equation and it can range from 0 to 1.

LSTMs (Long short term memory)

When we arrange our calendar for the day, we prioritize our appointments right? If in case we need to make some space for anything important we know which meeting could be canceled to accommodate a possible meeting.

Turns out that an RNN doesn't do so. In order to add new information, it transforms the existing information completely by applying a function. Because of this, the entire information is modified, on the whole, i. e. there is no consideration for '*important*' information and '*not so important*' information.

LSTMs on the other hand, make small modifications to the information by multiplications and additions. With LSTMs, the information flows through a mechanism known as cell states. This way, LSTMs can selectively remember or forget things. The information at a particular cell state has three different dependencies.

We'll visualize this with an example. Let's take the example of predicting stock prices for a particular stock. The stock price of today will depend upon:

1. The trend that the stock has been following in the previous days, may be a downtrend or an uptrend.
2. The price of the stock on the previous day, because many traders compare the stock's previous day price before buying it.
3. The factors that can affect the price of the stock for today. This can be a new company policy that is being criticized widely, or a drop in the company's profit, or maybe an unexpected change in the senior leadership of the company.

These dependencies can be generalized to any problem as:

1. The previous cell state (*i.e. the information that was present in the memory after the previous time step*)
2. The previous hidden state (*i.e. this is the same as the output of the previous cell*)
3. The input at the current time step (*i.e. the new information that is being fed in at that moment*)

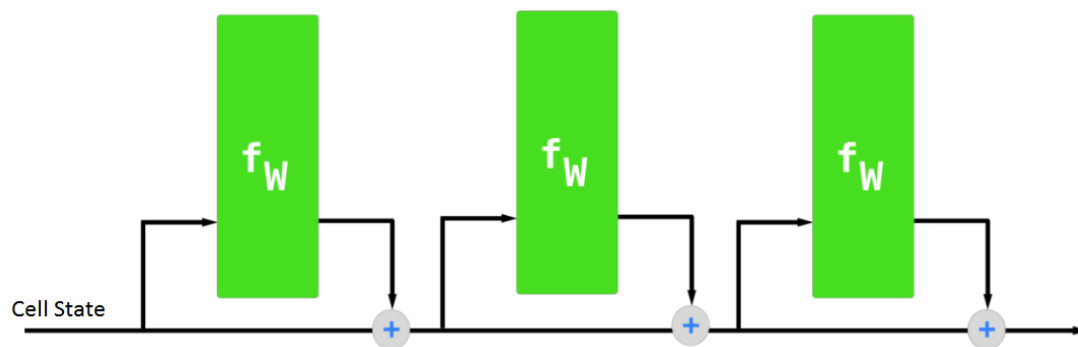
Another important feature of LSTM is its analogy with conveyor belts!

That's right!

Industries use them to move products around for different processes. LSTMs use this mechanism to move information around.

We may have some addition, modification, or removal of information as it flows through the different layers, just like a product may be molded, painted, or packed while it is on a conveyor belt.

The following diagram explains the close relationship between LSTMs and conveyor belts.



Although this diagram is not even close to the actual architecture of an LSTM, it solves our purpose for now.

Just because of this property of LSTMs, where they do not manipulate the entire information but rather modify them slightly, they are able to *forget* and *remember* things selectively.

Architecture of LSTMs

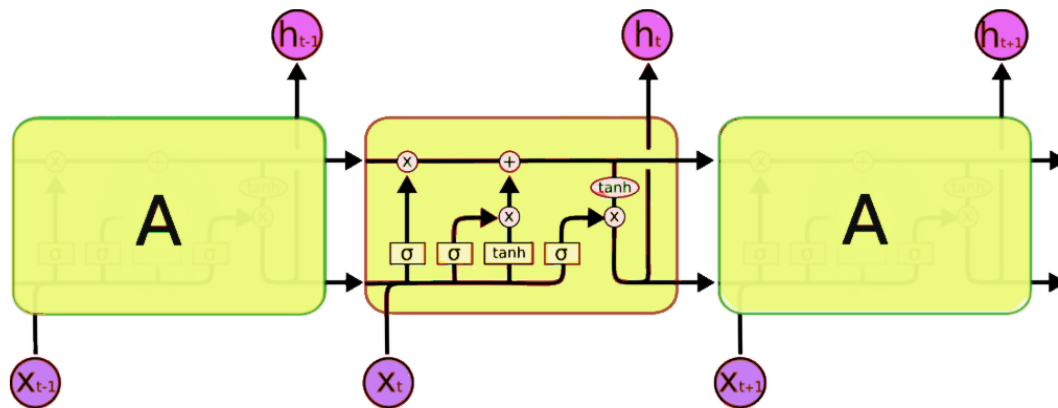
The functioning of LSTM can be visualized by understanding the functioning of a news channel's team covering a murder story. Now, a news story is built around facts, evidence, and statements of many people. Whenever a new event occurs you take either of the three steps.

Let's say, we were assuming that the murder was done by 'poisoning' the victim, but the autopsy report that just came in said that the cause of death was 'an impact on the head'. Being a part of this news team what do you do? You immediately forget the previous cause of death and all stories that were woven around this fact.

What, if an entirely new suspect is introduced into the picture. A person who had grudges with the victim and could be the murderer? You input this information into your news feed, right?

Now all these broken pieces of information cannot be served on mainstream media. So, after a certain time interval, you need to summarize this information and output the relevant things to your audience. Maybe in the form of "*XYZ turns out to be the prime suspect.*".

Now let's get into the details of the architecture of the LSTM network:



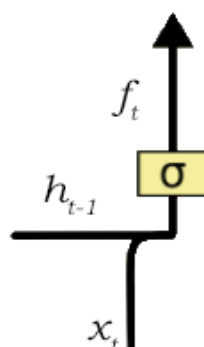
A typical LSTM network is comprised of different memory blocks called cells (the rectangles that we see in the image). There are two states that are being transferred to the next cell; the cell state and the hidden state. The memory blocks are responsible for remembering things and manipulations to this memory are done through three major mechanisms, called gates.

Forget Gate

Take the example of a text prediction problem. Let's assume an LSTM is fed in, the following sentence:

Bob is a nice person. Dan on the other hand is evil.

As soon as the first full stop after “*person*” is encountered, the forget gate realizes that there may be a change of context in the next sentence. As a result of this, the *subject* of the sentence is *forgotten* and the place for the subject is vacated. And when we start speaking about “*Dan*” this position of the subject is allocated to “*Dan*”. This process of forgetting the subject is brought about by the forget gate.



A forget gate is responsible for removing information from the cell state. The information that is no longer required for the LSTM to understand things or the information that is of less

importance is removed via the multiplication of a filter. This is required for optimizing the performance of the LSTM network.

This gate takes in two inputs; h_{t-1} and x_t .

h_{t-1} is the hidden state from the previous cell or the output of the previous cell and x_t is the input at that particular time step. The given inputs are multiplied by the weight matrices and a bias is added. Following this, the sigmoid function is applied to this value. The sigmoid function outputs a vector, with values ranging from 0 to 1, corresponding to each number in the cell state. Basically, the sigmoid function is responsible for deciding which values to keep and which to discard. If a '0' is output for a particular value in the cell state, it means that the forget gate wants the cell state to forget that piece of information completely. Similarly, a '1' means that the forget gate wants to remember that entire piece of information. This vector output from the sigmoid function is multiplied by the cell state.

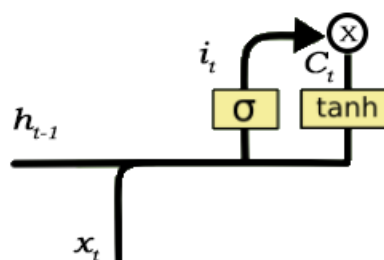
Input Gate

Okay, let's take another example where the LSTM is analyzing a sentence:

Bob knows swimming. He told me over the phone that he had served the navy for 4 long years.

Now the important information here is that "Bob" knows swimming and that he has served the Navy for four years. This can be added to the cell state, however, the fact that he told all this over the phone is a less important fact and can be ignored. This process of adding some new information can be done via the input gate.

Here is its structure:



The input gate is responsible for the addition of information to the cell state. This addition of information is basically a three-step process as seen from the diagram above.

1. Regulating what values need to be added to the cell state by involving a sigmoid function. This is basically very similar to the forget gate and acts as a filter for all the information from h_{t-1} and x_t .

2. Creating a vector containing all possible values that can be added (as perceived from h_{t-1} and x_t) to the cell state. This is done using the tanh function, which outputs values from -1 to +1.
3. Multiplying the value of the regulatory filter (the sigmoid gate) to the created vector (the tanh function) and then adding this useful information to the cell state via addition operation.

Once this three-step process is done with, we ensure that only that information is added to the cell state that is *important* and is not *redundant*.

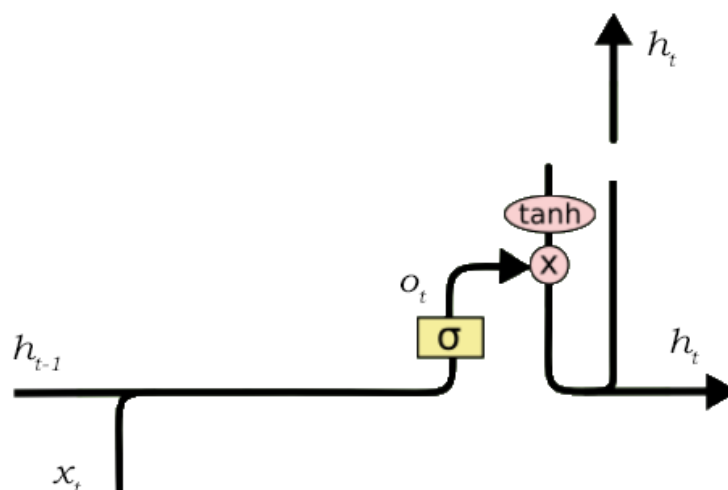
Output Gate

Not all information that runs along with the cell state, is fit for being output at a certain time. We'll visualize this with an example:

Bob fought single handedly with the enemy and died for his country. For his contributions brave ____.

In this phrase, there could be a number of options for the empty space. But we know that the current input of '*brave*', is an adjective that is used to describe a noun. Thus, whatever word follows, has a strong tendency of being a noun. And thus, Bob could be an apt output.

This job of selecting useful information from the current cell state and showing it out as output is done via the output gate. Here is its structure:



The functioning of an output gate can again be broken down to three steps:

1. Creating a vector after applying tanh function to the cell state, thereby scaling the values to the range -1 to +1.

2. Making a filter using the values of h_{t-1} and x_t , such that it can regulate the values that need to be output from the vector created above. This filter again employs a sigmoid function.
3. Multiplying the value of this regulatory filter to the vector created in step 1, and sending it out as a output and also to the hidden state of the next cell.

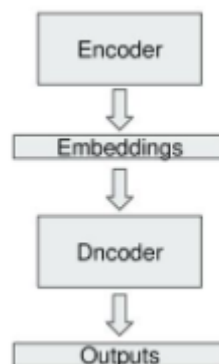
The filter in the above example will make sure that it diminishes all other values but '*Bob*'. Thus the filter needs to be built on the input and hidden state values and be applied on the cell state vector.

Encoder-Decoder Models

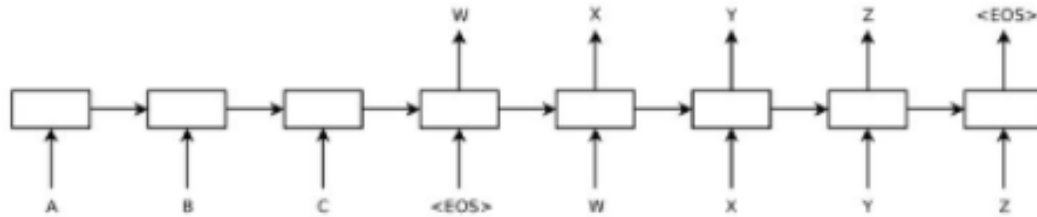
- The encoder-Decoder architecture was first introduced in the Statistical Machine Translation in 2014.
- It has become the state of art language model ever since then.
- It can be applied to many fields, we are interested in using it as a language model, together with RNN.
- Learning languages both semantically and syntactically.

Encoder-Decoder Structure

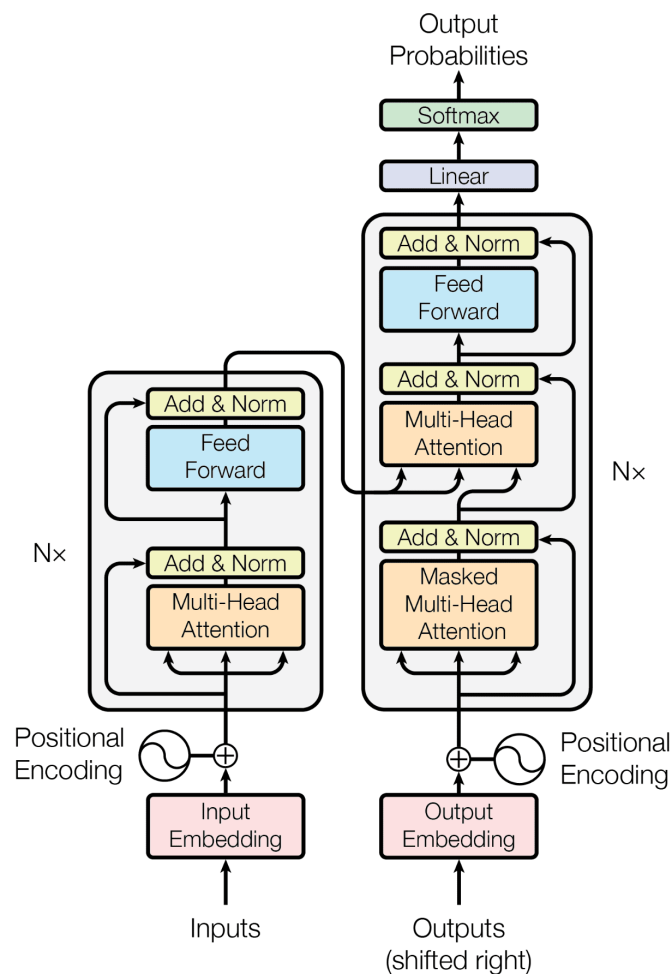
- The model consists of two RNNs - encoder, and decoder.
- Encoder maps the variable-length sequence to a fixed-length vector representation.
- Decoder maps the fixed-length vector representation back to a variable-length target sequence.
- Two networks are trained jointly to maximize the joint probability of the target sequence given a source sequence.



Encoder-decoder Model



- Input sequence feeds in at each timestep.
- The embedding is generated for each time step.
- The embedding is feeding into the decoder to generate the target sequence step by step.
- $p(y_1, \dots, y_T | x_1, \dots, x_T)$, conditional probability.
- Attention's usage in the encoder-decoder model.
- This is architecture, not a model. Transformer, Bert Model are all encoder-decoder models without touching RNN.
- The transformer as a purely linear network to process language.



Attention Mechanism

What is Attention?

In psychology, attention is the cognitive process of selectively concentrating on one or a few things while ignoring others.

A neural network is considered to be an effort to mimic human brain actions in a simplified manner. Attention Mechanism is also an attempt to implement the same action of selectively concentrating on a few relevant things, while ignoring others in deep neural networks.

Let me explain what this means. Let's say you are seeing a group photo of your first school. Typically, there will be a group of children sitting across several rows, and the teacher will sit somewhere in between. Now, if anyone asks the question, "How many people are there?", how will you answer it?

Simply by counting heads, right? You don't need to consider any other things in the photo. Now, if anyone asks a different question, "Who is the teacher in the photo?", your brain knows exactly what to do. It will simply start looking for the features of an adult in the photo. The rest of the features will simply be ignored. This is the 'Attention' which our brain is very adept at implementing.

How Attention Mechanism was Introduced in Deep Learning

The attention mechanism emerged as an improvement over the encoder decoder-based neural machine translation system in natural language processing (NLP). Later, this mechanism, or its variants, was used in other applications, including computer vision, speech processing, etc.

Before Bahdanau et al proposed the first Attention model in 2015, neural machine translation was based on encoder-decoder RNNs/LSTMs. Both encoder and decoder are stacks of LSTM/RNN units. It works in the two following steps:

1. The encoder LSTM is used to process the entire input sentence and encode it into a context vector, which is the last hidden state of the LSTM/RNN. This is expected to be a good summary of the input sentence. All the intermediate states of the encoder are ignored, and the final state is supposed to be the initial hidden state of the decoder
2. The decoder LSTM or RNN units produce the words in a sentence one after another

In short, there are two RNNs/LSTMs. One we call the encoder – this reads the input sentence and tries to make sense of it, before summarizing it. It passes the summary (context vector) to the decoder which translates the input sentence by just seeing it.

The main drawback of this approach is evident. If the encoder makes a bad summary, the translation will also be bad. And indeed it has been observed that the encoder creates a bad summary when it tries to understand longer sentences. It is called the long-range dependency problem of RNN/LSTMs.

RNNs cannot remember longer sentences and sequences due to the vanishing/exploding gradient problem. It can remember the parts which it has just seen. Even Cho et al (2014), who proposed the encoder-decoder network, demonstrated that the performance of the encoder-decoder network degrades rapidly as the length of the input sentence increases.

Although an LSTM is supposed to capture the long-range dependency better than the RNN, it tends to become forgetful in specific cases. Another problem is that there is no way to give more importance to some of the input words compared to others while translating the sentence.

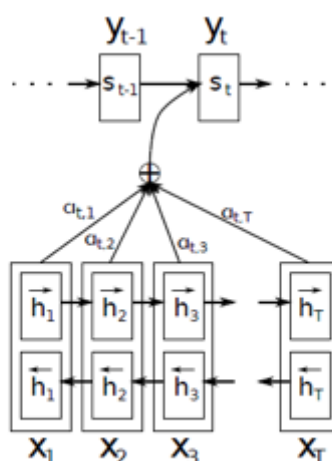
Now, let's say, we want to predict the next word in a sentence, and its context is located a few words back. Here's an example – “Despite originally being from Uttar Pradesh, as he was brought up in Bengal, he is more comfortable in Bengali”. In these groups of sentences, if we want to predict the word “Bengali”, the phrase “brought up” and “Bengal”- two should be given more weight while predicting it. And although Uttar Pradesh is another state's name, it should be “ignored”.

So is there any way we can keep all the relevant information in the input sentences intact while creating the context vector?

Bahdanau et al (2015) came up with a simple but elegant idea where they suggested that not only can all the input words be taken into account in the context vector, but relative importance should also be given to each one of them.

So, whenever the proposed model generates a sentence, it searches for a set of positions in the encoder hidden states where the most relevant information is available. This idea is called ‘Attention’.

Understanding the Attention Mechanism



This is the diagram of the Attention model shown in Bahdanau's paper. The Bidirectional LSTM used here generates a sequence of annotations ($h1, h2, \dots, hTx$) for each input

sentence. All the vectors $h_1, h_2, \dots$, etc., used in their work are basically the concatenation of forward and backward hidden states in the encoder.

$$h_j = \left[\overrightarrow{h}_j^T; \overleftarrow{h}_j^T \right]^T.$$

To put it in simple terms, all the vectors $h_1, h_2, h_3, \dots, h_{T_x}$ are representations of T_x number of words in the input sentence. In the simple encoder and decoder model, only the last state of the encoder LSTM was used (h_{T_x} in this case) as the context vector.

But Bahdanau et al put emphasis on embeddings of all the words in the input (represented by hidden states) while creating the context vector. They did this by simply taking a weighted sum of the hidden states.

Now, the question is how should the weights be calculated? Well, the weights are also learned by a feed-forward neural network and the mathematical equation is below.

The context vector c_i for the output word y_i is generated using the weighted sum of the annotations:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$

The weights α_{ij} are computed by a softmax function given by the following equation:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})},$$

$$e_{ij} = a(s_{i-1}, h_j)$$

e_{ij} is the output score of a feedforward neural network described by the function that attempts to capture the alignment between input at j and output at i .

Basically, if the encoder produces T_x number of “annotations” (the hidden state vectors) each having dimension d , then the input dimension of the feedforward network is $(T_x, 2d)$ (assuming the previous state of the decoder also has d dimensions and these two vectors are concatenated). This input is multiplied with a matrix W_a of $(2d, 1)$ dimensions (of course followed by addition of the bias term) to get scores e_{ij} (having a dimension $(T_x, 1)$).

On the top of these e_{ij} scores, a tan hyperbolic function is applied followed by a softmax to get the normalized alignment scores for output j :

$$E = I [T_x * 2d] * W_a [2d * 1] + B [T_x * 1]$$

$$\alpha = \text{softmax}(\tanh(E))$$

$$C = I T * \alpha$$

So, α is a $(T_x, 1)$ dimensional vector and its elements are the weights corresponding to each word in the input sentence.

Let α is $[0.2, 0.3, 0.3, 0.2]$ and the input sentence is “I am doing it”. Here, the context vector corresponding to it will be:

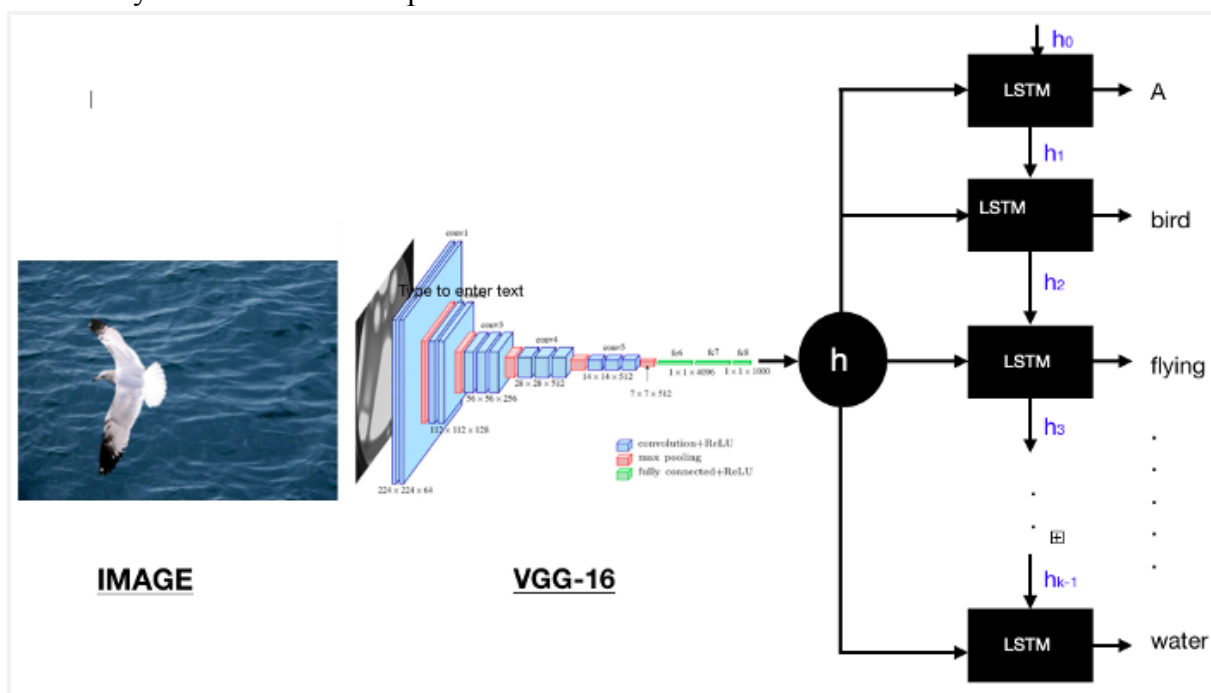
$$C = 0.2 * I^I + 0.3 * I^{\text{am}} + 0.3 * I^{\text{doing}} + 0.2 * I^{\text{it}} \quad [I_x \text{ is the hidden state corresponding to the word } x]$$

Attention over images

Caption generation is a challenging artificial intelligence problem where a textual description must be generated for a given photograph. It requires both methods from computer vision to understand the content of the image and a language model from the field of natural language processing to turn the understanding of the image into words in the right order.

A “**classic**” image captioning system would encode the image, using a pre-trained Convolutional Neural Network(**ENCODER**) that would produce a hidden state h .

Then, it would decode this hidden state by using an LSTM(**DECODER**) and generate recursively each word of the caption.



A classic image captioning model

Deep learning methods have demonstrated state-of-the-art results on caption generation problems. What is most impressive about these methods is a single end-to-end model can be defined to predict a caption, given a photo, instead of requiring sophisticated data preparation or a pipeline of specifically designed models.

Problem with ‘Classic’ Image Captioning Model

The problem with this method is that, when the model is trying to generate the next word of the caption, this word is usually describing only a part of the image. It is unable to capture the essence of the entire input image. Using the whole representation of the image h to condition the generation of each word cannot efficiently produce different words for different parts of the image. This is exactly where an **Attention mechanism** is helpful.

Concept of Attention Mechanism:

With an Attention mechanism, the image is first divided into n parts, and we compute with a Convolutional Neural Network (CNN) representations of each part $h_1, \dots, h_n$. When the RNN is generating a new word, the attention mechanism is focusing on the relevant part of the image, so the decoder only uses specific parts of the image.

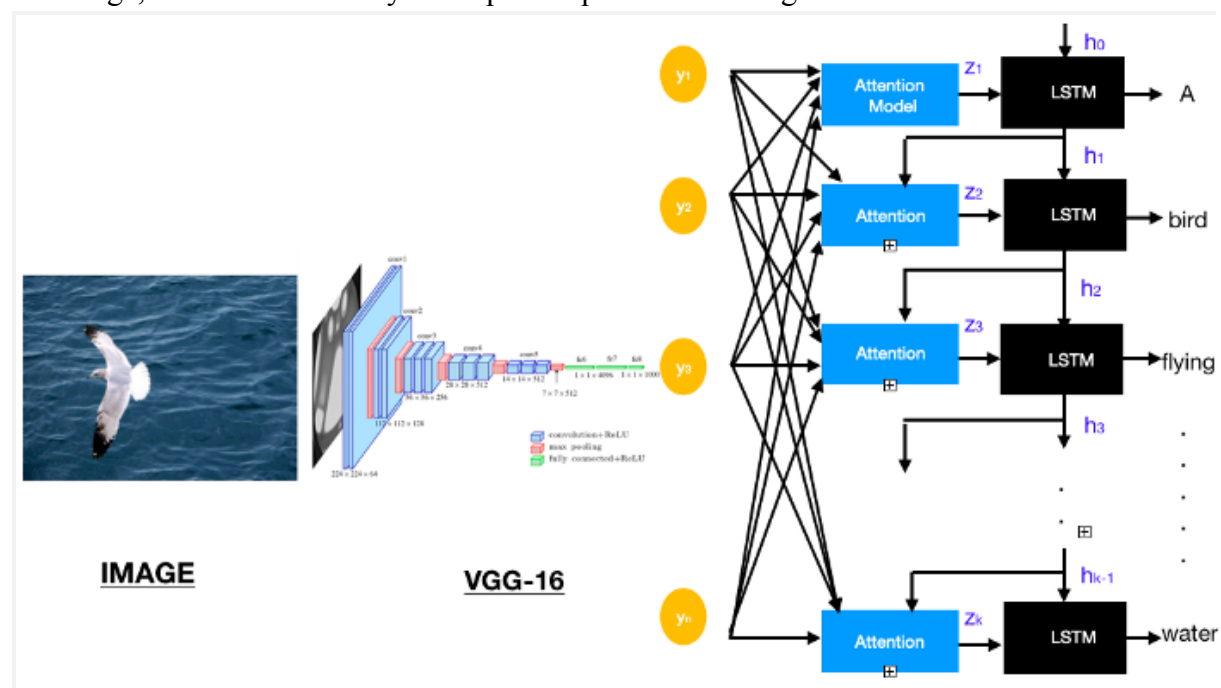


Image Captioning using Attention Mechanism

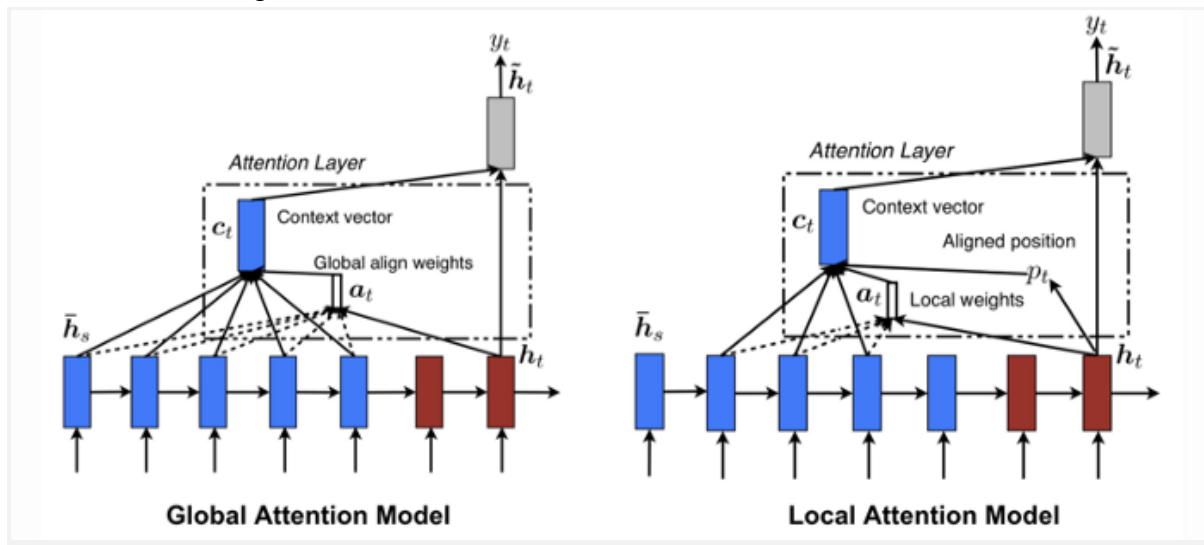
We can recognize the figure of the “classic” model for image captioning, but with a new layer of **attention model**. What is happening when we want to predict the new word of the caption? If we have predicted i words, the hidden state of the LSTM is h_i . We select the « relevant » part of the image by using h_i as the context. Then, the output of the attention model

z_i , which is the representation of the image filtered such that only the relevant parts of the image remains, is used as an input for the LSTM. Then, the LSTM predicts a new word and returns a new hidden state h_{i+1} .

Types of Attention Mechanism :

Attention could be broadly differentiated into 2 types:

1. **Global Attention(Luong's Attention):** Attention is placed on all source positions.
2. **Local Attention(Bahdanau Attention):** Attention is placed only on a few source positions.



Global vs Local Attention mechanisms

Both attention based models differ from the normal encoder-decoder architecture only in the decoding phase. These attention based methods differ in the way that they compute context vector ($c(t)$).

Few Explanations :

1.Global Attention

Global attention takes into consideration all encoder hidden states to derive the context vector ($c(t)$). In order to calculate $c(t)$, we compute $a(t)$ which is a variable length alignment vector. The alignment vector is derived by computing a similarity measure between $h(t)$ and $\bar{h}_{bar}(s)$ where $h(t)$ is the source hidden state while $\bar{h}_{bar}(s)$ is the target hidden state. Similar states in encoder and decoder are actually referring to the same meaning.

The score is a content-based function for which any of the following alternatives could have been used:

$$\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_s) = \begin{cases} \mathbf{h}_t^\top \bar{\mathbf{h}}_s & \text{dot} \\ \mathbf{h}_t^\top \mathbf{W}_a \bar{\mathbf{h}}_s & \text{general} \\ \mathbf{v}_a^\top \tanh(\mathbf{W}_a [\mathbf{h}_t; \bar{\mathbf{h}}_s]) & \text{concat} \end{cases}$$

The score function

Through score function, we are trying to calculate the similarity between the hidden states of the target and the source. Intuitively similar states in hidden and source refer to the same meaning but in different languages.

2. Local Attention

As Global attention focus on all source side words for all target words, it is computationally very expensive and is impractical when translating for long sentences. **To overcome this deficiency local attention chooses to focus only on a small subset of the hidden states of the encoder per target word.**

$$f_{ATT} = V_{attn}^T * \tanh(U_{attn} * h_j + W_{attn} * s_t)$$

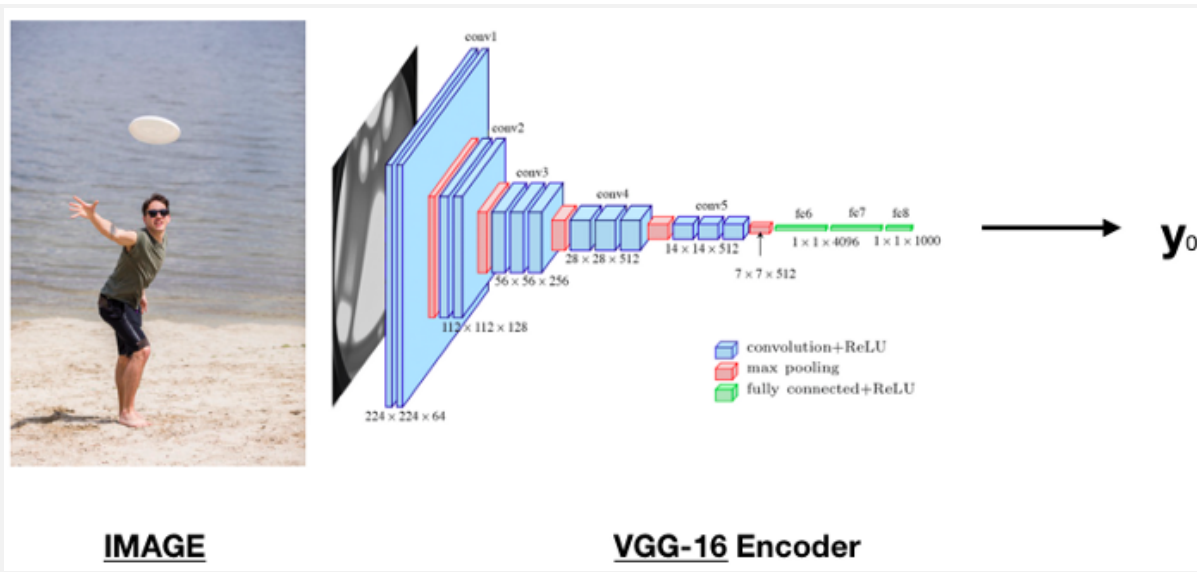
where,

$$V_{attn}^T \in R^d, U_{attn} \in R^{d*d}, W_{attn} \in R^{d*d}, s_{t-1} \in R^d, h_j \in R^d$$

Score for Local Attention

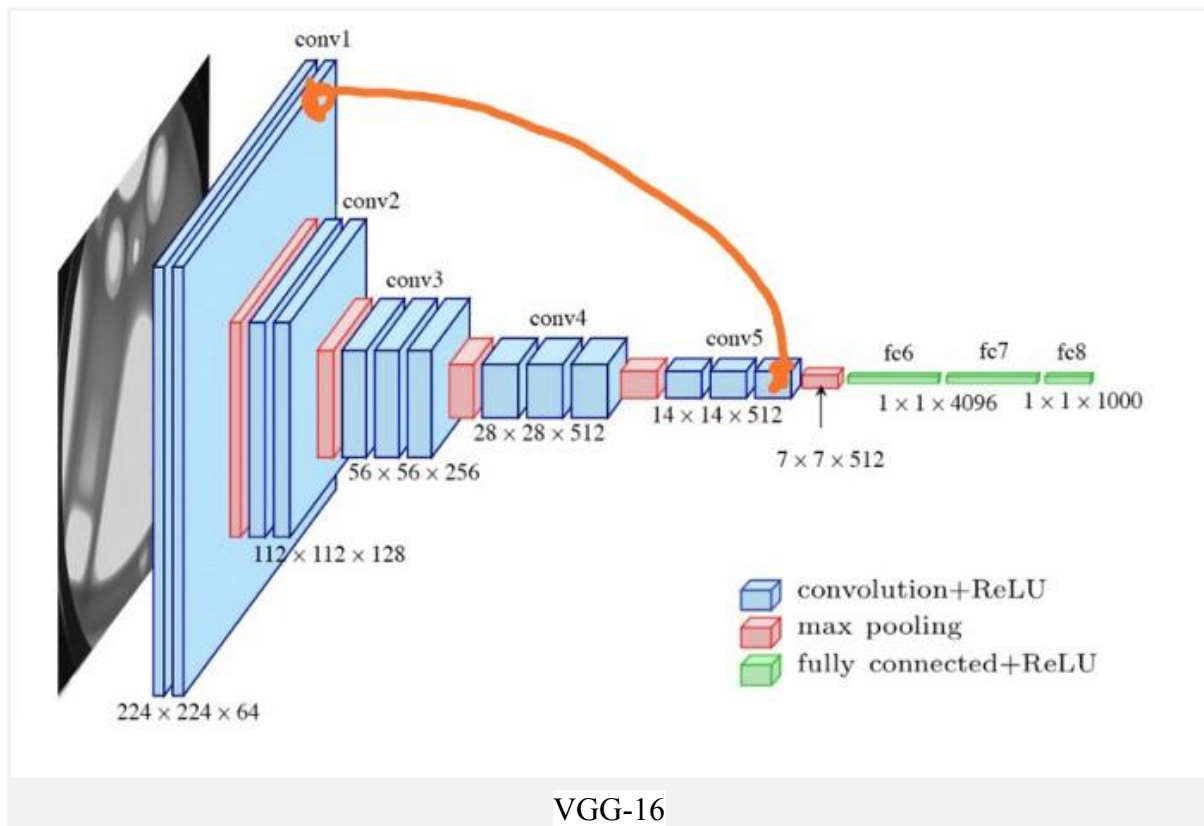
Let's discuss on how Attention Mechanism works

For images, we typically use representations from one of the fully connected layers. But suppose as shown in below figure, **a man is throwing a frisbee.**



So, when I say the word ‘**man**’ that means we need to focus only on man in the image ,and when I say the word ‘**throwing**’ then we have to focus on his hand in the image. Similarly , when we say ‘**frisbee**’ we have to focus only on the frisbee in the image. This means ‘**man**’, ‘**throwing**’ and ‘**frisbee**’ comes from different pixels in image. But the VGG-16 representation we used **does not contain any location information in it**.

But every location of convolution layers corresponds to some location of image as shown below.



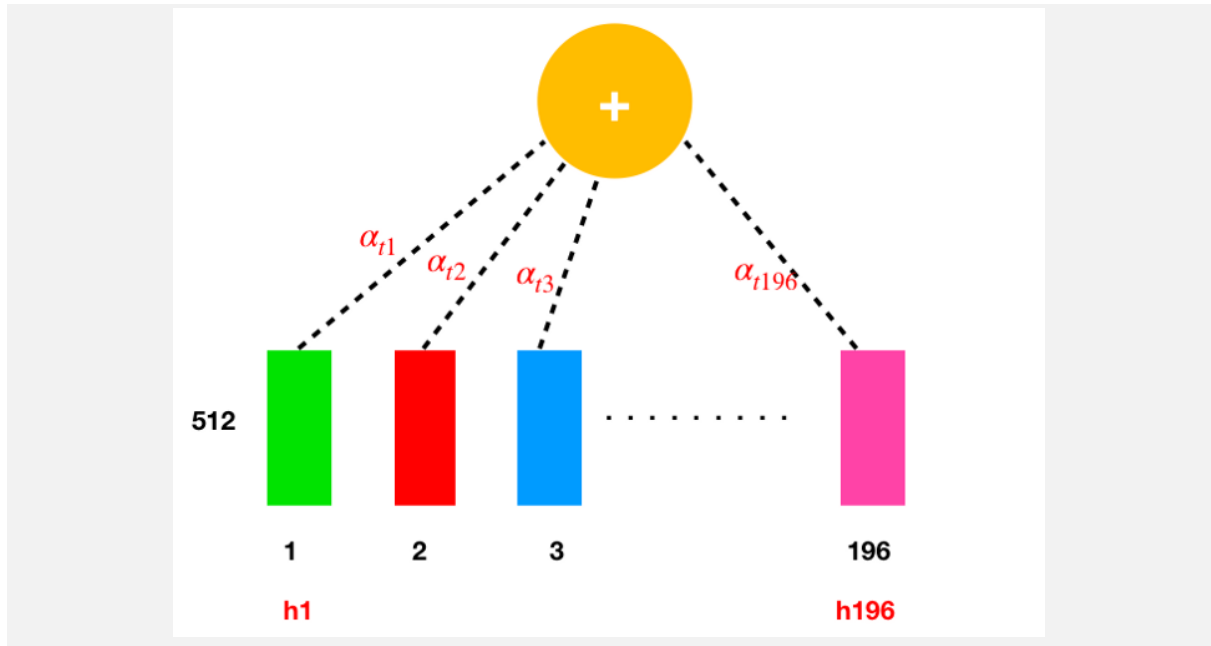
VGG-16

Now, for example, the output of the 5th convolution layer of VGGNet is a **$14 \times 14 \times 512$** size feature map.

This 5th convolution layer has 14×14 pixel locations which corresponds to certain portion in image, that means we have 196 such pixel locations.

And finally, we can treat these 196 locations(each having 512 dimensional representation) .

The model will then learn an attention over these locations(**which in turn corresponds to actual locations in the images**).



As shown in the above figure 5th convolution block is represented by 196 locations which can be passed in different time step.